



INSTITUTO POLITÉCNICO NACIONAL



Escuela Superior de Cómputo
Academia de ingeniería de software
Bases de Datos No Relacionales

Práctica IV:

Instalación y use de un sistema de bases de datos orientado a
columnas.

Profesora:

Rodríguez Sarabia Tania.

Contenido

1. Requisitos previos	2
2. Marco teórico	2
2.1 Bases de Datos Orientadas a Columnas	2
2.2 MonetDB	3
2.2.1 Aplicaciones de MonetDB.	4
3. Desarrollo de la práctica	5
4. Consultas	18
5. Actividad a realizar con Dataset Real: Video Game Sales	21

1. Requisitos previos

Para la realización de la práctica se deberá tener instalado en la computadora el software:

- [MonetDB](#).
- [DBeaver](#).
- [Driver de MonetDB para DBeaver](#).

2. Marco teórico

2.1 Bases de Datos Orientadas a Columnas

Las bases de datos tradicionales relacionales almacenan los datos en filas (row-based storage), lo que es útil para transacciones rápidas en aplicaciones OLTP (Online Transaction Processing). Sin embargo, para análisis de grandes volúmenes de datos, las bases de datos orientadas a columnas (column-based storage) ofrecen ventajas significativas en velocidad y compresión. Algunas de sus características son:

- Almacenamiento por columnas: Los datos de cada columna se almacenan juntos en disco, permitiendo accesos más rápidos a subconjuntos específicos de datos.
- Compresión eficiente: Como los datos de una columna suelen ser homogéneos, las técnicas de compresión son más efectivas en comparación con bases de datos relacionales tradicionales.
- Optimizado para consultas analíticas: En entornos OLAP (Online Analytical Processing), donde se requiere el análisis de grandes volúmenes de datos, este modelo permite mejorar drásticamente los tiempos de ejecución de consultas agregadas.
- Escalabilidad y paralelismo: Estas bases de datos están diseñadas para manejar consultas concurrentes y pueden distribuirse en arquitecturas de procesamiento paralelo.

2.2 MonetDB

MonetDB es un sistema de gestión de bases de datos (DBMS) de código abierto, diseñado para manejar grandes volúmenes de datos y optimizado para consultas analíticas. Fue desarrollado en el Centro de Informática de la Universidad de Amsterdam y ha sido

adoptado en diversos sectores para procesamiento de datos rápido y eficiente. Algunas de sus características son:

- Almacenamiento en columnas: MonetDB usa el almacenamiento orientado a columnas, lo que lo hace ideal para cargas de trabajo analíticas.
- Ejecución de consultas optimizada: Implementa técnicas de optimización avanzadas para acelerar el procesamiento de consultas SQL.
- Compatibilidad con SQL y otras interfaces: Soporta SQL, JDBC, ODBC y otros lenguajes de programación como Python y R.
- Procesamiento en memoria: MonetDB minimiza los accesos a disco al ejecutar consultas en memoria RAM.
- Integración con herramientas de análisis: Puede usarse con plataformas como DBEaver para administrar bases de datos de manera visual.

2.2.1 Aplicaciones de MonetDB.

Las bases de datos orientadas a columnas como MonetDB se utilizan en:

- Big Data y análisis de datos: Para consultas rápidas sobre grandes volúmenes de información.
- Business Intelligence: Soporte para reportes y análisis de negocios.

- Procesamiento de datos científicos: Donde se requiere manejar conjuntos de datos masivos con operaciones complejas.
- Sistemas financieros y bioinformática: Aplicaciones que requieren procesamiento rápido de grandes cantidades de datos transaccionales o experimentales.

3. Desarrollo de la práctica

- a) Para poder realizar la práctica es necesario descargar el software del sitio de MonetDB para la versión del Sistema Operativo que sea necesaria (MacOS, Linux o Windows). Ejecutar el instalador tal y como se muestra en la figura 1:



Figura 1. Setup wizard de MonetDB.

- b) La siguiente figura muestra el acuerdo de licencia del software. Es necesario aprobar para poder continuar.



Figura 2. Acuerdo de licencia de MonetDB.

- c) En la pantalla siguiente se debe elegir el tipo de instalación: típica, personalizada y completa. Se recomienda no modificar y continuar mediante el botón Next, para la versión típica.

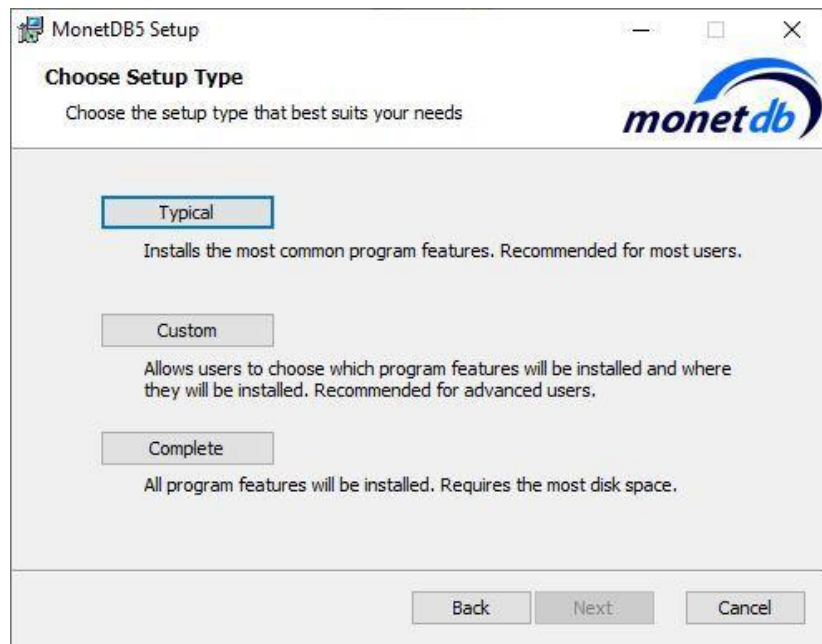


Figura 3. Tipos de instalaciones para MonetDB.

- d) En la siguiente pantalla (figura 4), se debe dar click en el botón Install para comenzar la instalación. Hay que considerar que se deben de dar permisos en Windows de ejecución.

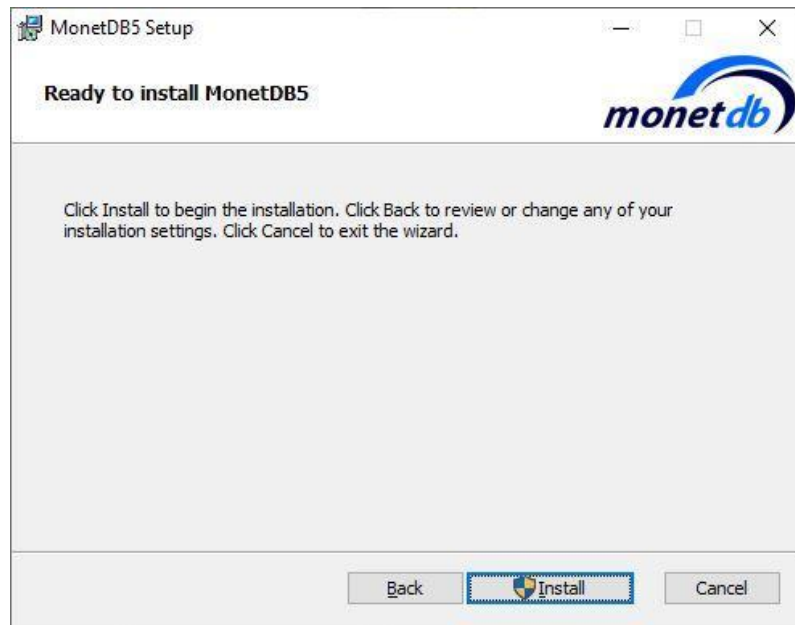


Figura 4. Inicio de instalación del software.

- e) Una vez concluida la instalación, se crea un grupo de programas (MonetDB) en el menú del sistema de Windows, con varios íconos descriptivos de las actividades que se pueden realizar con la base de datos. Abra una ventana del Sistema Operativo (en caso de Windows debe elegir la opción de ejecutar como Administrador), y localice la ruta en disco C:\Program Files\MonetDB\MonetDB5\bin.

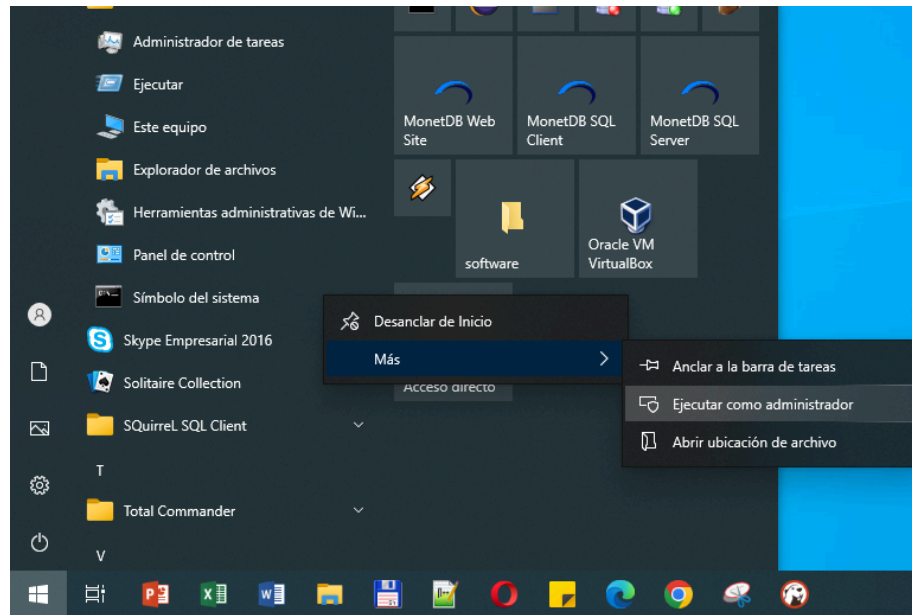


Figura 5. Ejecución del servidor.

Una vez realizado esto, ejecute en la ventana los siguientes comandos, una vez finalizado, no cierre la ventana.

```
C:\>mkdir empresa
C:\>cd C:\Program Files\MonetDB\MonetDB5\bin
C:\Program Files\MonetDB\MonetDB5\bin>mserver5
--dbpath=c:\empresa
```

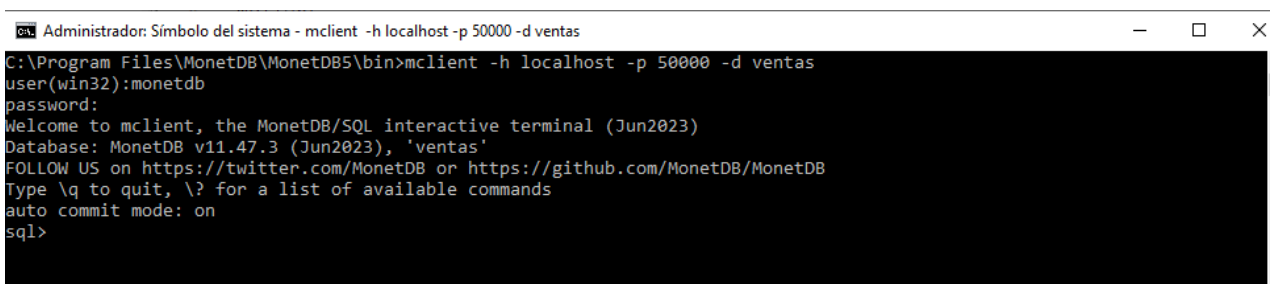
```
Administrador: Símbolo del sistema - mserver5.exe --dbpath=c:\empresa
C:\Program Files\MonetDB\MonetDB5\bin>mserver5.exe --dbpath=c:\empresa
# MonetDB 5 server v11.47.3 (Jun2023)
# Serving database 'empresa', using 4 threads
# Compiled for amd64-pc-windows-msvc/64bit
# Found 7.991 GiB available main-memory of which we use 6.512 GiB
# Copyright (c) 1993 - July 2008 CWI.
# Copyright (c) August 2008 - 2023 MonetDB B.V., all rights reserved
# Visit https://www.monetdb.org/ for further information
# MonetDB/GIS module loaded
# MonetDB/SQL module loaded
# Listening for connection requests on mapi:monetdb://localhost:50000/
# MonetDB server is started. To stop server press Ctrl-C.
```

Figura 6. Servidor en ejecución.

- f) Ahora abra una nueva ventana de línea de comandos SQL, siguiendo el mismo procedimiento anterior, y escriba las siguientes sentencias:


```
C:\>cd C:\Program Files\MonetDB\MonetDB5\bin
C:\Program Files\MonetDB\MonetDB5\bin>mclient -h
localhost -p 50000 -d empresa
```

Posteriormente, ingrese los valores de usuario (user): *monetdb* y contraseña (password): *monetdb*.



```
Administrador: Símbolo del sistema - mclient -h localhost -p 50000 -d ventas
C:\Program Files\MonetDB\MonetDB5\bin>mclient -h localhost -p 50000 -d ventas
user(win32):monetdb
password:
Welcome to mclient, the MonetDB/SQL interactive terminal (Jun2023)
Database: MonetDB v11.47.3 (Jun2023), 'ventas'
FOLLOW US on https://twitter.com/MonetDB or https://github.com/MonetDB/MonetDB
Type \q to quit, \? for a list of available commands
auto commit mode: on
sql>
```

Figura 7. Línea de comandos SQL.

En la terminal escriba los siguientes comandos:

- CREATE USER "gerente" WITH PASSWORD 'gerente' NAME 'gerente' SCHEMA "sys";
- CREATE SCHEMA "empresa" AUTHORIZATION "gerente";
- ALTER USER "gerente" SET SCHEMA "empresa";
- SELECT * FROM SYS.SCHEMAS;
- SELECT * FROM SYS.USERS;
- EXIT

g) Inicie el software DBeaver y cree una conexión con el servidor de MonetDB.

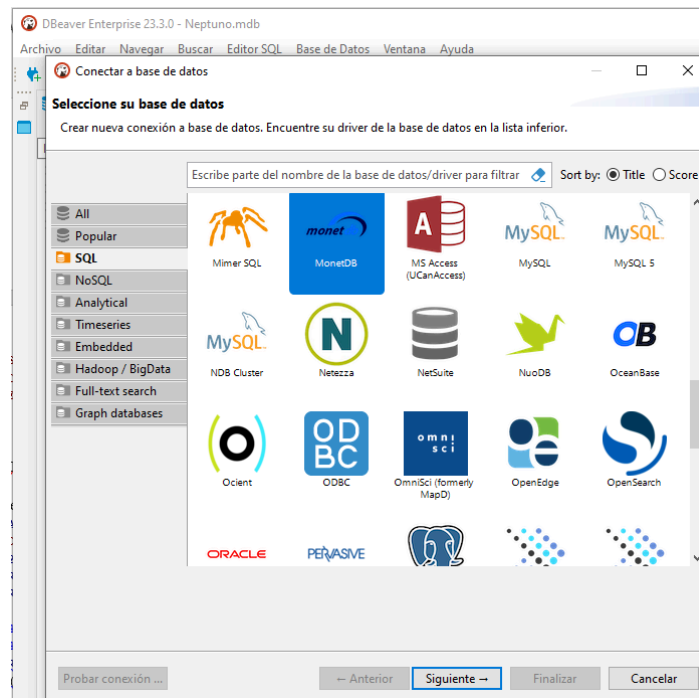


Figura 8. Selección de la BD a usar.

h) Escriba los parámetros de conexión mostrados en la figura 9 (contraseña *gerente*).

Conectar a base de datos

Conexión genérica JDBC
MonetDB ajustes de conexión

General Driver properties SSH + Network configurations...

General

Connect by: ☒ Host ☐ URL Driver type: JDBC

JDBC URL: jdbc:monetdb://localhost:50000/empresa

Host: localhost Port: 50000

Database/Schema: empresa

Authentication

Authentication: Database Native

Nombre de usuario: gerente

Contraseña: ☒ Save password

[You can use variables in connection parameters.](#) Connection details (name, type, ...)

Driver name: MonetDB Driver Settings

Probar conexión ... Anterior Siguiente Finalizar Cancelar

Figura 9. Credenciales de acceso.

Nota: En caso de que el driver de MonetDB no se encuentre en DBeaver, usar el driver anexado en la sección de Requisitos previos y agregarlo en la sección *driver properties* mostrada en la figura anterior.

- i) Expanda el árbol de conexión y seleccione la opción de empresa, como se observa en la figura 10 y oprima la tecla F3 para abrir el editor de comandos SQL:

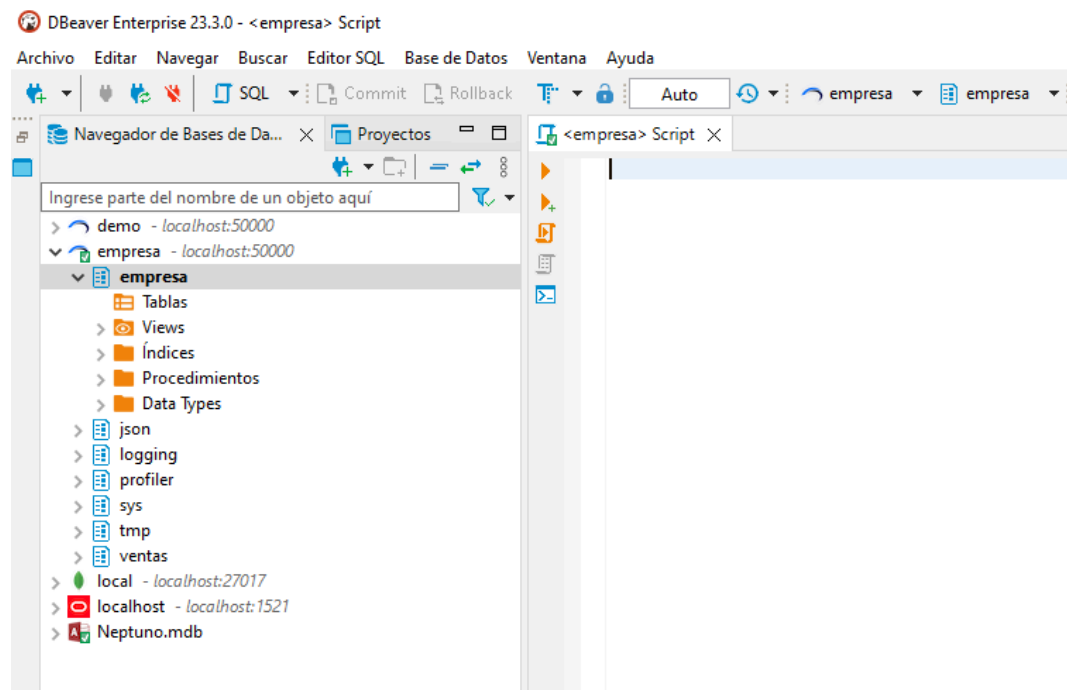


Figura 10. Selección de la bd "empresa".

Posteriormente, en el editor, ejecute las siguientes sentencias:

```
1. CREATE TABLE EMPLEADO (
    NSS CHAR(10) NOT NULL,
    NOMBRE VARCHAR2(30) NOT NULL,
    PATERNO VARCHAR2(45) NOT NULL,
    MATERNO VARCHAR2(45) NOT NULL,
    FECHA_NAC DATE NOT NULL,
    CALLE VARCHAR(40) NOT NULL,
    NUMERO NUMBER(4),
    COLONIA VARCHAR(40) NOT NULL,
    CIUDAD VARCHAR(30) NOT NULL,
    CODPOS CHAR(5),
    GENERO CHAR(1) NOT NULL,
    SALARIO NUMBER(7,2) DEFAULT 2000.00,
    JEFE CHAR(10),
    CARGO VARCHAR(50),
    OFICINA NUMBER(5,0),
    TITULO VARCHAR2(20),
    FECHA_CONTRATACION DATE NOT NULL,
    CONSTRAINT EMP_GENERO_CHK CHECK (UPPER(GENERO) = 'M' OR
    UPPER(GENERO) = 'F'),
    CONSTRAINT EMP_SAL_CHK CHECK (SALARIO BETWEEN 2500.0 AND
    50000.0),
    CONSTRAINT EMP_NSS_PK PRIMARY KEY (NSS));
```

```

2. ALTER TABLE EMPLEADO ADD CONSTRAINT EMP_EMP_FK FOREIGN KEY (JEFE)
   REFERENCES EMPLEADO (NSS) DISABLE;

3. CREATE TABLE OFICINA (
   NUM_OF NUMBER(3,0),
   NOMBRE_OF VARCHAR2(30) NOT NULL,
   JEFE_OF CHAR(10) NOT NULL,
   CONSTRAINT OF_NUM_PK PRIMARY KEY (NUM_OF),
   CONSTRAINT OF_NOMBRE_PK UNIQUE (NOMBRE_OF));

4. ALTER TABLE EMPLEADO ADD CONSTRAINT EMP_OF_FK FOREIGN KEY
   (OFICINA) REFERENCES OFICINA (NUM_OF) DISABLE;

5. ALTER TABLE OFICINA ADD CONSTRAINT OF_EMP_FK FOREIGN KEY
   (JEFE_OF) REFERENCES EMPLEADO (NSS) DISABLE;

6. CREATE TABLE UBICACION_OF (
   NUM_OF NUMBER(2,0) NOT NULL,
   UBIC_OF VARCHAR2(40),
   CONSTRAINT NUM_OF_UBIC_PK PRIMARY KEY (NUM_OF, UBIC_OF));

7. ALTER TABLE UBICACION_OF ADD CONSTRAINT UBIC_OF_FK FOREIGN KEY
   (NUM_OF) REFERENCES OFICINA (NUM_OF) DISABLE;

8. CREATE TABLE DEPENDIENTE (
   NOMBRE VARCHAR2(40),
   PATERNO VARCHAR2(45) NOT NULL,
   MATERNO VARCHAR2(45) NOT NULL,
   FECHA_NAC DATE NOT NULL,
   PARENTESCO VARCHAR2(20) NOT NULL,
   NSS_EMP CHAR(10) NOT NULL,
   GENERO CHAR(1) NOT NULL,
   CONSTRAINT DTE_NOMBRE_NSS_PK PRIMARY KEY (NOMBRE, NSS_EMP),
   CONSTRAINT DTE_GENERO_CHK CHECK (UPPER(GENERO) = 'M' OR
   UPPER(GENERO) = 'F'),
   CONSTRAINT DTE_PARENTESCO_CHK CHECK (UPPER(PARENTESCO) IN
   ('HIJO', 'HIJA', 'PADRE', 'MADRE', 'NIETA', 'NIETO')));

9. ALTER TABLE DEPENDIENTE ADD CONSTRAINT DEP_EMP_FK FOREIGN KEY
   (NSS_EMP) REFERENCES EMPLEADO (NSS) DISABLE;

10. CREATE TABLE PROYECTO (
   NUM_PROY NUMBER(6,0) NOT NULL,
   NOMBRE_PROY VARCHAR2(80) NOT NULL,
   DESCRIPCION VARCHAR(300),
   FECHA_INICIO DATE NOT NULL,
   FECHA_ENTREGA DATE,
   PRESUPUESTO NUMBER(12,2) DEFAULT 50000.00,
   ESTADO VARCHAR2(20) NOT NULL,
   NUM_OF NUMBER(3,0),
   CONSTRAINT PROJ_PK PRIMARY KEY (NUM_PROY),
   CONSTRAINT PROJ_UK UNIQUE (NOMBRE_PROY),
   CONSTRAINT PROJ_ESTADO_CHK CHECK (UPPER(ESTADO) IN ('APROBADO',
   'RECHAZADO', 'CANCELADO', 'EJECUTANDO', 'ENTREGADO',
   'MODIFICADO')),

```

- ```

 CONSTRAINT PROJ_PRESUP_CHK CHECK (PRESUPUESTO BETWEEN 50000.00
 AND 500000.00));

11. ALTER TABLE PROYECTO ADD CONSTRAINT PROJ_OF_FK FOREIGN KEY
 (NUM_OF) REFERENCES OFICINA(NUM_OF) DISABLE;

12. CREATE TABLE TRABAJA (
 NSS CHAR(10) NOT NULL ,
 NUM_PROY NUMBER(6,0) NOT NULL ,
 HORAS NUMBER(4,0) NOT NULL ,
 CONSTRAINT TRAB_PK PRIMARY KEY (NSS, NUM_PROY));

13. ALTER TABLE TRABAJA ADD CONSTRAINT TRAB_PROY_FK FOREIGN KEY
 (NUM_PROY) REFERENCES PROYECTO (NUM_PROY) DISABLE;

14. ALTER TABLE TRABAJA ADD CONSTRAINT TRAB_EMP_FK FOREIGN KEY
 (NSS) REFERENCES EMPLEADO (NSS) DISABLE;

```

Continúe con la inserción de datos, ejecutando las siguientes sentencias:

- ```

15.    INSERT INTO EMPLEADO
        (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
        ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACI
        ON) VALUES (777888999,'Jesús','López','López',
        TO_DATE('04/04/1973','dd/mm/yyyy'),'Calle Venus', 45,
        'Irrigación', 'Ecatepec', '89765','M',50000,NULL,'Administrador
        Técnico',3,'Ingeniero', TO_DATE('05/04/2000','dd/mm/yyyy'));

16.    INSERT INTO EMPLEADO
        (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
        ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACI
        ON) VALUES (222333444,'Guadalupe','Oñate','Martínez',
        TO_DATE('24/11/1969','dd/mm/yyyy'),'Av.Revolución', 348,
        'Fuentes', 'San Ignacio',
        '67656','F',25000,777888999,'Oficinista',3,'Licenciado',
        TO_DATE('07/04/2000','dd/mm/yyyy'));

17.    INSERT INTO EMPLEADO
        (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
        ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACI
        ON) VALUES (444555666,'Julia','Regalado','Rodríguez',
        TO_DATE('30/07/1975','dd/mm/yyyy'),'Calle cielo', 76,
        'Independencia', 'Tepeji',
        '34256','F',28000,111222333,'Desarrollador',1,'Ingeniero',
        TO_DATE('28/06/2001','dd/mm/yyyy'));

18.    INSERT INTO EMPLEADO
        (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
        ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACI
        ON) VALUES (555666777,'Mario','Medina','Morales',
        TO_DATE('01/10/1977','dd/mm/yyyy'),'Av.Politécnico', 650,
        'Profesiones', 'CDMX',
        '09765','M',20000,777888999,'Documentador',3,'Licenciado',
        TO_DATE('30/05/2000','dd/mm/yyyy'));

```

```

19.  INSERT INTO EMPLEADO
      (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
      ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACION)
      VALUES (333444555,'Rogelio','Calzada','Rubio',
      TO_DATE('25/03/1965','dd/mm/yyyy'),'Av.Independencia', 123,
      'Fuentes', 'San Ignacio', '56565','M',39000,NULL,'Gerente
      general',2,'Licenciado', TO_DATE('05/04/2000','dd/mm/yyyy'));

20.  INSERT INTO EMPLEADO
      (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
      ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACION)
      VALUES (666777888,'Bruce','Bolaños','Zepeda',
      TO_DATE('07/08/1963','dd/mm/yyyy'),'Mar Mediterráneo', 56, 'Lomas
      Lindas', 'CDMX',
      '09879','M',24000,111222333,'Oficinista',1,'Licenciado',
      TO_DATE('17/12/2000','dd/mm/yyyy'));

21.  INSERT INTO EMPLEADO
      (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
      ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACION)
      VALUES (999000111,'Laura','Méndez','López',
      TO_DATE('20/02/1982','dd/mm/yyyy'),'Av.Montevideo', 98,
      'Ampliación Hidalgo', 'Texcoco',
      '45679','F',18000,333444555,'Desarrollador',2,'Ingeniero',
      TO_DATE('20/12/2000','dd/mm/yyyy'));

22.  INSERT INTO EMPLEADO
      (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
      ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACION)
      VALUES (111222333,'Sandra','Guzmán','Martínez',
      TO_DATE('27/11/1970','dd/mm/yyyy'),'Av.Siempreviva', 444, 'San
      Mateo', 'Ecatepec', '89740','F',45000,NULL,'Director de
      Operaciones',1,'Ingeniero', TO_DATE('20/12/2000','dd/mm/yyyy'));

23.  INSERT INTO EMPLEADO
      (NSS,NOMBRE,PATERNO,MATERNO,FECHA_NAC,CALLE,NUMERO,COLONIA,CIUDAD
      ,CODPOS,GENERO,SALARIO,JEFE,CARGO,OFICINA,TITULO,FECHA_CONTRATACION)
      VALUES (888999000,'Guadalupe','Hidalgo','Pérez',
      TO_DATE('19/03/1985','dd/mm/yyyy'),'Av.Miguel Hidalgo', 56,
      'Independencia', 'Tepeji',
      '87981','F',27000,333444555,'Desarrollador',2,'Ingeniero',
      TO_DATE('11/10/2001','dd/mm/yyyy'));

24.  ALTER TABLE EMPLEADO ENABLE CONSTRAINT EMP_EMP_FK;

25.  INSERT INTO OFICINA(NUM_OF,NOMBRE_OF,JEFE_OF) VALUES
      (1,'Sistemas', 111222333);

26.  INSERT INTO OFICINA(NUM_OF,NOMBRE_OF,JEFE_OF) VALUES
      (2,'Ventas', 333444555);

27.  INSERT INTO OFICINA(NUM_OF,NOMBRE_OF,JEFE_OF) VALUES
      (3,'Administración', 777888999);

28.  ALTER TABLE EMPLEADO ENABLE CONSTRAINT EMP_OF_FK;

```

```

29.  ALTER TABLE OFICINA ENABLE CONSTRAINT OF_EMP_FK;

30.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES
      ('Monterrey',3);

31.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES
      ('Guadalupe',3);

32.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES ('Juárez',3);

33.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES ('DF',1);

34.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES ('DF',2);

35.  INSERT INTO UBICACION_OF(UBIC_OF,NUM_OF) VALUES ('Pachuca',2);

36.  ALTER TABLE UBICACION_OF ENABLE CONSTRAINT UBIC_OF_FK;

37.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Gustavo','Guzmán','Ramírez',TO_DATE('07/12/1999','dd/mm/yyyy'),
      'HIJO',111222333,'M');

38.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Karen','Oñate','Rodríguez',TO_DATE('09/11/1995','dd/mm/yyyy'),'
      HIJA',222333444,'F');

39.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Julieta','Castillo','López',TO_DATE('09/11/1951','dd/mm/yyyy'),
      'MADRE',333444555,'F');

40.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Lucia','Calzada','Castillo',TO_DATE('09/11/1995','dd/mm/yyyy'),
      'HIJA',333444555,'F');

41.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Ernesto','Medina','Medina',TO_DATE('28/10/1945','dd/mm/yyyy'),'
      PADRE',555666777,'M');

42.  INSERT INTO
      dependiente(NOMBRE,PATERNO,MATERNO,FECHA_NAC,PARENTESCO,NSS_EMP,G
      ENERO) VALUES
      ('Carlos','López','Pérez',TO_DATE('12/04/2005','dd/mm/yyyy'),'HIJ
      O',777888999,'M');

```



```

43.  INSERT INTO
    dependiente (NOMBRE, PATERNO, MATERNO, FECHA_NAC, PARENTESCO, NSS_EMP, G
ENERO) VALUES
    ('Karla', 'Hidalgo', 'Serrano', TO_DATE('26/08/2009', 'dd/mm/yyyy'), '
HIJA', 888999000, 'F');

44.  INSERT INTO
    dependiente (NOMBRE, PATERNO, MATERNO, FECHA_NAC, PARENTESCO, NSS_EMP, G
ENERO) VALUES
    ('Sergio', 'Méndez', 'Carrillo', TO_DATE('17/12/1992', 'dd/mm/yyyy'),
'HIJO', 999000111, 'M');

45.  INSERT INTO
    dependiente (NOMBRE, PATERNO, MATERNO, FECHA_NAC, PARENTESCO, NSS_EMP, G
ENERO) VALUES
    ('Felipe', 'Bolaños', 'Hernández', TO_DATE('26/08/2009', 'dd/mm/yyyy'
), 'NIETO', 666777888, 'M');

46.  INSERT INTO
    dependiente (NOMBRE, PATERNO, MATERNO, FECHA_NAC, PARENTESCO, NSS_EMP, G
ENERO) VALUES
    ('Rodrigo', 'Calzada', 'Castillo', TO_DATE('21/07/1997', 'dd/mm/yyyy'
), 'HIJO', 333444555, 'M');

47.  ALTER TABLE DEPENDIENTE ENABLE CONSTRAINT DEP_EMP_FK;

48.  INSERT INTO
    proyecto (NOMBRE_PROY, NUM_PROY, NUM_OF, FECHA_INICIO, FECHA_ENTREGA, D
ESCRIPCION, ESTADO, PRESUPUESTO) VALUES ('Sitio
Web', 2, 1, TO_DATE('18/02/2010', 'dd/mm/YYYY'), NULL, 'Desarrollo de
un sitio web corporativo para la empresa Productos Entregables
SA', 'EJECUTANDO', 375000.00);

49.  INSERT INTO
    proyecto (NOMBRE_PROY, NUM_PROY, NUM_OF, FECHA_INICIO, FECHA_ENTREGA, D
ESCRIPCION, ESTADO, PRESUPUESTO) VALUES ('Mejoramiento de
Calidad', 1, 2, TO_DATE('10/01/2012', 'dd/mm/YYYY'), NULL, 'Propuesta
de implementación de un sistema de software para la vigilancia y
reporte de los productos con baja calida y/o devueltos por
clientes', 'APROBADO', NULL);

50.  INSERT INTO
    proyecto (NOMBRE_PROY, NUM_PROY, NUM_OF, FECHA_INICIO, FECHA_ENTREGA, D
ESCRIPCION, ESTADO, PRESUPUESTO) VALUES
    ('Publicidad', 3, 3, TO_DATE('06/09/2011', 'dd/mm/YYYY'),
TO_DATE('29/08/2012', 'dd/mm/YYYY'), 'Sistema de control de la
ubicación de espectaculares publicitarios en diversas zonas de la
CDMX', 'ENTREGADO', 298000.00);

51.  ALTER TABLE PROYECTO ENABLE CONSTRAINT PROY_OF_FK;

52.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
    (222333444, 3, 8);

53.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
    (555666777, 2, 16);

```

```

54.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
      (888999000, 2, 8);

55.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
      (555666777, 1, 10);

56.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
      (666777888, 1, 2);

57.  INSERT INTO trabaja (NSS, NUM_PROY, HORAS) VALUES
      (888999000, 1, 8);

58.  ALTER TABLE TRABAJA DISABLE CONSTRAINT TRAB_PROY_FK;

59.  ALTER TABLE TRABAJA DISABLE CONSTRAINT TRAB_EMP_FK;

60.  SELECT OBJECT_NAME, OBJECT_TYPE, CREATED, STATUS FROM
      USER_OBJECTS ORDER BY OBJECT_TYPE;

61.  SELECT TABLE_NAME, TABLESPACE_NAME, STATUS, NESTED, READ_ONLY
      FROM USER_TABLES;

62.  SELECT
      COLUMN_NAME, DATA_TYPE, DATA_LENGTH, DATA_PRECISION, DATA_SCALE, NULLA
      BLE FROM USER_TAB_COLUMNS WHERE TABLE_NAME = 'EMPLEADO';

63.  SELECT
      CONSTRAINT_NAME, CONSTRAINT_TYPE, SEARCH_CONDITION, R_CONSTRAINT_NAM
      E, DELETE_RULE, STATUS FROM USER_CONSTRAINTS WHERE TABLE_NAME =
      'EMPLEADO';

```

Verifique que en la ejecución de las sentencias anteriores sea correcta.

4. Consultas

Ejecute las siguientes sentencias. Si en el ambiente gráfico tiene dificultades, puede emplear el ícono de Run SQL Command Line del grupo de programa en la instalación de Oracle. Observe el carácter / como terminador de sentencias, que únicamente tiene sentido en la terminal de comandos SQL de Oracle (Run SQL Command Line).

```

64.  CREATE OR REPLACE FUNCTION ultima_oficina RETURN NUMBER IS
    num_o oficina.num_of%TYPE := 0;
    BEGIN
        SELECT max(num_of) INTO num_o FROM oficina;
        RETURN num_o;
    END;
    /

65.  SELECT ultima_oficina FROM dual;

66.  CREATE OR REPLACE PROCEDURE inserta_oficina(nombre VARCHAR,
jefe CHAR) IS
    num_of oficina.num_of%TYPE := ultima_oficina;
    BEGIN
        IF nombre IS NULL OR jefe IS NULL THEN
            DBMS_OUTPUT.PUT_LINE('Los datos del departamento y del
jefe son obligatorios. ');
        ELSIF num_of IS NOT NULL THEN
            INSERT INTO oficina (NUM_OF,NOMBRE_OF,JEFE_OF) VALUES
(num_of + 1,nombre,jefe);
            IF SQL%ROWCOUNT > 0 THEN
                COMMIT;
            ELSE
                RAISE_APPLICATION_ERROR(-20001,'No fue posible
insertar el registro');
            END IF;
        END IF;
    END;
    /

68.BEGIN
    inserta_oficina('Ventas', 666777888);
END;
/

69.BEGIN
    inserta_oficina('Recursos Humanos', null);
END;
/

70.BEGIN
    inserta_oficina('Recursos Humanos', 666777888);
END;
/

71.SELECT * FROM oficina;

72.SELECT OBJECT_NAME, OBJECT_TYPE, CREATED, STATUS FROM USER_OBJECTS
ORDER BY OBJECT_TYPE;

```

```

73.SELECT NAME, LINE, TEXT FROM USER_SOURCE;

74.CREATE OR REPLACE FUNCTION antiguedad(fecha_cont DATE) RETURN
NUMBER IS
    res NUMBER := 0;
    BEGIN
        res := EXTRACT(YEAR FROM SYSDATE) - EXTRACT(YEAR FROM
fecha_cont);
    RETURN res;
    END;
/

75.SELECT nss, nombre || ' ' || paterno || ' ' || materno as
nombre_completo, antiguedad(fecha_contratacion) as antiguedad FROM
empleado ORDER BY antiguedad DESC;

76.CREATE OR REPLACE FUNCTION genera_fecha(anio INTEGER, mes INTEGER,
dia INTEGER, sep VARCHAR) RETURN DATE IS
patron CHAR(12) := ' ' || 'YYYY' || sep || 'mm' || sep || 'dd' ||
' ';
fecha CHAR(12) := ' ' || anio || sep || mes || sep || dia || ' ';
d DATE := null;
BEGIN
    d := TO_DATE(fecha, patron);
    RETURN d;
END;
/

77.DECLARE dia DATE := genera_fecha(2000, 4, 8, '/');
BEGIN
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(dia, 'YYYY/mm/dd'));
END;
/

78.CREATE OR REPLACE FUNCTION edad(fecha_nac DATE) RETURN NUMBER IS
res NUMBER := 0;
BEGIN
    res := MONTHS_BETWEEN(SYSDATE, fecha_nac);
    RETURN ROUND(res/12,0);
END;
/

79.SELECT nombre || ' ' || paterno || ' ' || materno as
nombre_completo, edad(fecha_nac) as edad, parentesco FROM
dependiente ORDER BY edad;

80.SELECT OBJECT_NAME, OBJECT_TYPE, CREATED, STATUS FROM USER_OBJECTS
ORDER BY OBJECT_TYPE;

```

5. Actividad a realizar con Dataset Real: Video Game Sales

Para esta sección, trabajaremos con el dataset "Video Game Sales", el cual contiene una lista de videojuegos con ventas superiores a las 100,000 copias, incluyendo detalles sobre la plataforma, el año de lanzamiento, el género, la editorial y el desglose de ventas por región.

Una vez que hayas importado el archivo CSV a tu base de datos en MonetDB como una tabla llamada **vgsales** a través de DBeaver, abre el editor de comandos SQL y resuelve las siguientes actividades utilizando las sentencias de consulta y agregación de SQL.

Link de descarga: <https://www.kaggle.com/datasets/gregorut/videogamesales>

Actividad 1: Mostrar todos los juegos cuyo género (Genre) sea "Sports" y hayan sido lanzados en el año 2006, ordenados por sus ventas globales (Global_Sales) de mayor a menor.

Actividad 2: Listar las 10 plataformas (Platform) con la mayor suma total de ventas en Norteamérica (NA_Sales).

Actividad 3: Listar los juegos que contienen la palabra "Mario" en su nombre (Name) usando el operador LIKE y proyectar en pantalla únicamente el nombre, la plataforma, el año y sus ventas globales.

Actividad 4: Buscar todos los juegos que fueron publicados por la empresa "Nintendo" (Publisher igual a "Nintendo").

Actividad 5: Encontrar los juegos que vendieron más de 5 millones de copias en Europa (EU_Sales) y menos de 1 millón de copias en Japón (JP_Sales).

Actividad 6: Contar cuántos juegos se han lanzado históricamente por cada tipo de género (Genre) usando la cláusula GROUP BY.

Actividad 7: Obtener el promedio de ventas en Japón (JP_Sales) agrupado por la empresa publicadora (Publisher).

Actividad 8: Mostrar las plataformas (Platform) que tienen más de 1,000 juegos registrados en la base de datos (requiere usar GROUP BY y luego la cláusula HAVING).

Actividad 9: Listar las editoriales (Publisher) que lograron obtener la distinción de superar los 50 millones de copias acumuladas en ventas globales (Global_Sales) sumando todos sus títulos.

Actividad 10: Calcular el año de lanzamiento (Year) máximo y mínimo registrado, agrupado por Plataforma (Platform).

Actividad 11: Por cada empresa publicadora (Publisher), mostrar en un solo reporte:

- Total de juegos publicados.
- Promedio de ventas globales (Global_Sales).
- Total acumulado de ventas en el resto del mundo (Other_Sales).

Actividad 12: Identificar el Top 5 de géneros (Genre) con la mayor cantidad total de copias vendidas globalmente, filtrando y excluyendo previamente a los juegos que son de la plataforma "PC".

Actividad 13: Obtener los años de lanzamiento (Year) en los que la suma total de ventas en Europa (EU_Sales) fue superior a la suma total de ventas en Norteamérica (NA_Sales).

Actividad 14: Obtener el promedio de ventas globales (Global_Sales) por género (Genre), filtrando la consulta inicial para ignorar aquellos registros que no tengan el año de lanzamiento registrado (Year IS NOT NULL).

Actividad 15: Calcular el porcentaje estimado que representan las ventas de Japón respecto a las ventas globales ($JP_Sales / Global_Sales * 100$) para cada juego, limitando el reporte a los 20 videojuegos con el porcentaje más alto, y asegurando que las ventas globales sean mayores a 1 millón para evitar sesgos numéricos.

